

AI TESTER

GitHub Code Reviewer Agent with Custom UI

Phase-by-Phase Prompt Roadmap

A copy-paste implementation guide for an AI coding assistant

UI -> GitHub File URL -> n8n Webhook -> AI Code Review -> Formatted UI Result

Organization	AI Tester
Frontend	Plain HTML + CSS + JavaScript
n8n Webhook	http://localhost:5678/webhook/Github_code_reviewer
HTTP Method	GET, query parameter: url
Deployment	Localhost; no authentication

How to Use This Roadmap

- Complete the phases sequentially. Each phase deliberately extends the application created in the earlier phases instead of rebuilding it.
- Copy only the “Copy-Paste Prompt for AI Coding Assistant” from the current phase into your coding AI. Run and test the result before moving to the next phase.
- The frontend remains an interface and integration layer. It must not implement a second code-review engine because review intelligence already exists in n8n.
- If a phase exposes an integration problem, fix that phase first. Do not proceed while the browser verification steps are failing.

Confirmed Integration Contract

Item	Confirmed behavior
Application brand	AI Tester. No fixed logo/colour palette was supplied, so the UI should use a professional GitHub/developer-oriented visual theme while keeping organization name, logo treatment, colours, and key visual tokens easy to configure.
User input	A GitHub file URL in the established browser URL format, for example: <code>https://github.com/testleafrainings/Sel_May_2026/blob/main/Week5/Day2/LearnDragAndDrop.java</code>
Backend endpoint	<code>http://localhost:5678/webhook/Github_code_reviewer</code>
Request	GET request. Pass the user-entered GitHub URL as the query parameter named <code>url</code> .
Response	The webhook returns the generated code-review result as review text/Markdown. Do not assume additional JSON fields.
Authentication	None.
Hosting	Run the frontend locally and provide a localhost URL/instructions for the user to open in a browser.

Responsibility Boundary

Frontend / Custom UI

- Collect and validate the GitHub file URL.
- Call the existing n8n webhook.

- Expose waiting, loading, success, validation, and error states.
- Receive and safely present the returned review.
- Allow the user to perform another review.

Existing n8n Workflow

- Receive the URL from the webhook request.
- Retrieve/process the GitHub code.
- Pass the retrieved code through the configured Basic LLM Chain and AI chat model.
- Generate the review result.
- Return the review through Respond to Webhook.

Target User Journey

Enter GitHub URL -> Review Code -> Review in Progress -> Review Completed -> Display Code Review

Phase 1: Application Foundation and Configurable Design System

Phase Goal

Create a clean localhost-ready HTML/CSS/JavaScript application foundation, establish a configurable visual design system, and prepare the project for later phases without adding n8n integration yet.

What Will Be Implemented

- Create the basic static application structure using plain HTML, CSS, and JavaScript only.
- Establish centralized brand/design variables so colours and visual identity can be changed later without rewriting the UI.
- Create an initial application shell with a neutral professional GitHub/developer-oriented look for AI Tester.
- Provide a simple local serving method and tell the user the exact localhost URL to open.
- Do not build code-review logic or call n8n in this phase.

Copy-Paste Prompt for AI Coding Assistant

Continue by creating the foundation for a new web application named “GitHub Code Reviewer Agent” for the organization “AI Tester”.

Technology constraints:

- Use only plain HTML, CSS, and vanilla JavaScript.
- Do not introduce React, Vue, Angular, Node frameworks, build pipelines, or a backend server unless one is strictly needed only to serve the static files locally.
- The application will run locally in a browser.

Create a clear, maintainable project structure and a professional developer-oriented design system. AI Tester does not yet have a supplied logo asset or fixed brand palette, so create a tasteful GitHub/developer-inspired visual direction, but keep the organization name, logo treatment, colours, spacing, border radius, typography, and other major theme tokens centralized and easy to change later.

At this stage, build only the application foundation and overall page shell. Include an appropriate page title, sensible semantic structure, and a central main content area that later phases can extend.

Keep accessibility and responsive behavior in mind from the beginning. Use readable contrast, visible keyboard focus states, semantic HTML, and avoid unnecessary animation.

Important boundaries:

- Do not implement code-review intelligence in the frontend.
- Do not call n8n yet.
- Do not invent API fields, response structures, credentials, or authentication.
- Do not hardcode the sample GitHub URL as application data.

After implementing the foundation, give me simple instructions for running the site locally and state the exact localhost URL I should open in my browser. Preserve this project for the next phases rather than treating this as a throwaway prototype.

User-Facing Verification / Testing Steps

- ☐ Run the local serving command provided by the coding assistant.
- ☐ Open the stated localhost URL and confirm the page loads without a console error.
- ☐ Resize the browser and confirm the page foundation does not overflow horizontally.
- ☐ Inspect the code briefly and confirm major visual values are centralized rather than scattered through many selectors.
- ☐ Confirm there is no n8n request and no fake code-review result yet.

Expected Result

A stable local web project exists with a configurable AI Tester design system and a clean foundation that later prompts can extend.

Phase 2: AI Tester Branding, Header, Footer, and Professional Layout

Phase Goal

Build the permanent branded application frame so users immediately understand the product and organization before the interactive review workspace is added.

What Will Be Implemented

- Add an AI Tester identity area with a simple configurable logo treatment.
- Create a professional header with product name and concise purpose statement.
- Create a polished footer with organization/product context.
- Establish the main page hierarchy and developer-tool aesthetic without unrelated dashboard metrics or navigation features.
- Keep desktop, tablet, and mobile behavior clean.

Copy-Paste Prompt for AI Coding Assistant

Extend the existing application from Phase 1. Do not rebuild or replace the project structure.

Create the permanent branded frame for the application.

Branding:

- Organization name: AI Tester.
- Product name: GitHub Code Reviewer Agent.
- No official logo file or exact brand colours were provided. Create a simple, professional logo treatment that can later be replaced by an actual logo asset without restructuring the header.
- Continue using a refined GitHub/developer-oriented theme and keep the brand colours configurable through the existing design tokens.

Header:

- Add a professional application header containing the AI Tester identity and the product name.
- Include a concise subtitle or helper text that clearly communicates the purpose: users submit a GitHub code-file URL and receive an AI-generated code review through the existing automation workflow.
- Avoid adding fake connection indicators, user avatars, analytics, navigation pages, history links, API docs links, or features that do not exist.

Main layout:

- Create a focused single-purpose layout suitable for a developer tool.
- Reserve a prominent area for the review form and a separate review-results workspace that will be

implemented in later phases.

- The layout should feel polished but not visually cluttered.

Footer:

- Add a professional footer using AI Tester branding and a short product descriptor.
- Do not add legal links or company information that was not supplied.

Make the header/footer and page frame responsive on desktop, tablet, and mobile. Preserve semantic HTML, keyboard accessibility, and clear focus styles.

User-Facing Verification / Testing Steps

- ☐ Confirm “AI Tester” and “GitHub Code Reviewer Agent” are clearly visible.
- ☐ Confirm the interface feels like a focused developer tool, not a Jira dashboard or generic analytics dashboard.
- ☐ Verify no fabricated user profile, connection status, history, metrics, or navigation features appear.
- ☐ Resize to tablet and mobile widths; confirm header, content, and footer remain readable and aligned.
- ☐ Verify the logo treatment can be replaced later without changing the rest of the layout.

Expected Result

The application has a polished, organization-branded frame with clear product identity and space prepared for the review interaction.

Phase 3: GitHub File URL Review Workspace

Phase Goal

Create the primary user interaction: entering a GitHub file URL and initiating a review, while still using a simulated UI transition rather than calling n8n.

What Will Be Implemented

- Add a labeled GitHub file URL input with example format guidance.
- Add a prominent “Review Code” action and Enter-key support.
- Create an initial results/waiting workspace explaining what the user should do.
- Add a “review another file” reset pathway in the UI model.
- Do not connect the real webhook yet.

Copy-Paste Prompt for AI Coding Assistant

Extend the application created in Phases 1 and 2. Keep the existing branding and layout.

Build the main GitHub code-review workspace.

Input area:

- Add a clearly labelled input for a GitHub code-file URL.
- The expected style of input is a browser GitHub file URL such as:
`https://github.com/testleaftrainings/Sel_May_2026/blob/main/Week5/Day2/LearnDragAndDrop.java`
- Treat that URL only as an example shown in helper/placeholder text. Do not automatically submit it and do not hardcode it as the application’s real value.
- Add concise helper text explaining that the user should paste a direct GitHub file URL.
- Add a prominent “Review Code” button.
- Allow Enter in the input to trigger the same action once validation exists.

Review workspace:

- Create a large results panel with a friendly default/waiting state.
- The waiting state should explain the simple journey: Enter GitHub URL -> Review Code -> Review in Progress -> Review Completed.
- Reserve enough space to display a long structured code review later, including headings, lists, tables, inline code, and code blocks.

Interaction for this phase:

- Wire the controls only enough to demonstrate the transition between the default workspace and a temporary local “ready for integration” state.

- Do not call the n8n webhook yet.
- Do not generate a fake AI review.
- Add a clean reset/“Review Another File” interaction pattern that later phases can reuse after a completed review.

Keep the page focused, professional, responsive, and accessible.

User-Facing Verification / Testing Steps

- ☐ Paste the provided example GitHub URL and confirm it fits cleanly in the input.
- ☐ Press Enter and click “Review Code”; confirm both can trigger the same local interaction state.
- ☐ Confirm no network request is sent to n8n yet.
- ☐ Confirm the default workspace clearly tells a first-time user what to do.
- ☐ Use “Review Another File” or reset behavior and confirm the UI returns to the initial state.

Expected Result

The complete user-facing review workspace exists and the interaction flow is visually understandable, but backend integration is intentionally still absent.

Phase 4: Validation and Complete UI State Model

Phase Goal

Implement reliable client-side validation and the full set of visible application states before introducing the real network request.

What Will Be Implemented

- Validate empty, malformed, and unsupported GitHub file URL input.
- Create idle, validation-error, loading, success-ready, request-error, and reset states.
- Disable duplicate submission while a review is in progress.
- Provide clear status text and accessible live feedback.
- Keep validation aligned with the confirmed GitHub file URL format without limiting to one programming language.

Copy-Paste Prompt for AI Coding Assistant

Extend the existing application. Do not connect n8n yet. In this phase, make the form and review workspace behavior robust by implementing the complete UI state model.

Validation requirements:

- Empty input must show a clear required-field message.
- Parse the value as a URL instead of relying only on a loose text pattern.
- Require HTTPS.
- Require the host to be github.com.
- Require a direct GitHub file URL in the established form containing an owner, repository, /blob/, branch/ref, and file path.
- Do not restrict the file extension to Java; the UI is a GitHub code reviewer and should not silently reject other code-file extensions unless the backend contract later requires it.
- Trim accidental leading/trailing whitespace.
- Do not accept a repository home page as if it were a direct file URL.

State model:

1. Idle/default state: form available and waiting panel visible.
2. Validation-error state: input remains enabled, invalid field is visually identified, and a useful message explains what to fix.
3. Loading state: input and Review Code action are temporarily disabled to prevent duplicate submissions; show “Review in Progress” and a professional processing indicator.
4. Success-ready state: create the state container that later phases will fill with the actual returned review.
5. Request-error state: create a reusable error presentation with a retry path.

6. Reset state: allow the user to review another file and return the interface to a clean idle state.

Accessibility:

- Associate error text with the input.
- Move or announce important status changes appropriately using accessible live regions where practical.
- Keep keyboard focus logical when an error occurs or when a review completes.

For now, the submit action may briefly demonstrate the loading state and then return to a neutral “integration pending” state. Do not fabricate review content and do not call any endpoint yet.

User-Facing Verification / Testing Steps

- ☐ Click “Review Code” with the input empty; verify a visible, understandable validation message.
- ☐ Try text that is not a URL; verify it is rejected without a browser crash.
- ☐ Try a non-GitHub URL; verify it is rejected.
- ☐ Try a GitHub repository homepage; verify the UI asks for a direct file URL.
- ☐ Try the valid example GitHub file URL; verify the loading state disables duplicate submission and shows “Review in Progress.”
- ☐ Use keyboard-only navigation and verify focus/validation remain understandable.

Expected Result

The frontend now has a predictable and testable state machine, so introducing the real webhook in the next phase will not require redesigning the interaction.

Phase 5: Real n8n Webhook Integration

Phase Goal

Connect the validated GitHub file URL to the existing production n8n webhook using the confirmed GET contract and safely receive the response.

What Will Be Implemented

- Use the exact production webhook URL and GET method.
- Send the GitHub file URL as the query parameter named url.
- Construct the request using URL/URLSearchParams so the code-file URL is correctly encoded.
- Treat successful response content as review text/Markdown, not as an invented JSON schema.
- Integrate network lifecycle with the Phase 4 loading/error states.

Copy-Paste Prompt for AI Coding Assistant

Extend the existing application from Phases 1-4 and now connect it to the existing n8n workflow.

Use this confirmed integration contract exactly:

- n8n production webhook: `http://localhost:5678/webhook/Github_code_reviewer`
- HTTP method: GET
- Query parameter name: url
- The value of url is the validated GitHub file URL entered by the user.
- Example final request shape:
`http://localhost:5678/webhook/Github_code_reviewer?url=https://github.com/testleaftrainings/Sel_May_2026/blob/main/Week5/Day2/LearnDragAndDrop.java`
- No authentication is required.
- The n8n workflow is already responsible for dynamically using the supplied URL, retrieving the GitHub code, passing the code to its LLM chain/model, generating the review, and responding through the webhook.
- The successful response should be treated as code-review text/Markdown. Do not invent JSON response fields.

Implementation behavior:

1. Run the Phase 4 validation before any request is sent.
2. On valid submission, switch immediately to the existing loading state and prevent duplicate requests.
3. Build the GET request safely using URL and searchParams/URLSearchParams rather than manually concatenating an unescaped user URL.
4. Send the request with the browser Fetch API.
5. Check HTTP success/failure explicitly.
6. Read a successful response as text.

7. Pass that returned text into a dedicated result-rendering function/container. A later phase will improve the formatting.
8. Restore the form and UI correctly when the request succeeds or fails.

Do not:

- Add a second code reviewer in JavaScript.
- Call GitHub directly from the frontend as a replacement for n8n.
- Change the request to POST.
- Rename the query parameter.
- Create fake credentials or headers.
- Assume JSON response fields.

Keep the webhook URL in one clear configuration location so it can be changed later without searching through the entire codebase.

If the browser reports a CORS or mixed-environment issue, surface it as a clear integration error and explain the observed browser/network problem. Do not silently redesign the architecture or add an unrequested proxy.

User-Facing Verification / Testing Steps

- ☐ Make sure the n8n workflow is active and available at the production webhook URL.
- ☐ Open Browser Developer Tools -> Network. Submit the valid example GitHub file URL.
- ☐ Confirm exactly one GET request is sent to `/webhook/Github\_code\_reviewer`.
- ☐ Inspect the request query string and verify the parameter name is `url` and the full GitHub file URL is correctly encoded as its value.
- ☐ Confirm the UI remains in “Review in Progress” while waiting.
- ☐ Confirm the n8n response is received by the browser and the returned text is passed to the result area.
- ☐ Temporarily stop n8n and submit again; verify the page shows a controlled request error instead of crashing.

Expected Result

The custom UI is now functionally connected to the real n8n workflow using the confirmed production GET request contract.

Phase 6: Safe Markdown Review Rendering and Result Presentation

Phase Goal

Turn the returned n8n review text into a polished, readable code-review report without depending on a fabricated response schema.

What Will Be Implemented

- Create a dedicated result view for long reviews.
- Render common Markdown structures such as headings, emphasis, lists, inline code, fenced code blocks, separators, and tables.
- Safely escape untrusted content and avoid unsafe direct HTML injection.
- Gracefully fall back when Markdown is incomplete or malformed.
- Show completion status and a clear “Review Another File” action.

Copy-Paste Prompt for AI Coding Assistant

Extend the connected application from Phase 5. Do not change the webhook contract.

Improve how the n8n code-review response is presented.

Known response behavior:

- The successful response is review text/Markdown.
- It may contain content such as a title, overall score, section headings, bullet lists, numbered priorities, bold text, inline code, fenced code, horizontal separators, and Markdown tables.
- Do not assume named JSON fields such as score, readability, maintainability, or recommendations. The rendering must work from the returned text itself.

Create a safe Markdown-style result renderer suited for the sample code-review format.

Presentation requirements:

- Display the review inside a clear “Code Review Result” workspace.
- Give the top title and major section headings strong hierarchy.
- Make scores and important emphasis easy to scan without attempting to infer unsupported structured fields.
- Style bullet lists and numbered lists cleanly.
- Render inline code and code blocks with a readable monospace treatment.
- Render valid Markdown tables as responsive styled tables.

- If a table or other Markdown fragment is malformed, do not break the page. Fall back to a readable text/preformatted presentation for that fragment.
- Preserve meaningful line breaks.
- Long content should remain comfortable to read on desktop and mobile.

Security requirement:

- Treat the returned webhook content as untrusted text.
- Escape HTML before applying any supported Markdown transformation, or use another approach that prevents arbitrary HTML/script execution.
- Do not blindly place the raw response into innerHTML.

Completion behavior:

- Replace “Review in Progress” with “Review Completed”.
- Use a restrained success indicator.
- Keep the reviewed GitHub URL visible in a compact, readable form so the user knows what was reviewed.
- Provide a clear “Review Another File” action that resets the input, errors, status, and old result.

Do not add fake review sections or fabricate a score if the n8n response does not contain one.

User-Facing Verification / Testing Steps

- ☐ Submit the example GitHub file URL and wait for the real n8n review.
- ☐ Confirm headings, bold text, lists, inline code, and separators are visually distinct.
- ☐ If the response includes a Markdown table, confirm it is readable and scrolls or adapts safely on small screens.
- ☐ Confirm malformed Markdown does not make the result disappear or break the layout.
- ☐ Confirm raw HTML/script-like text from a response would be displayed safely rather than executed.
- ☐ Click “Review Another File” and verify the old result is cleared and the input is ready again.

Expected Result

The application now presents the actual n8n review as a polished, safe, readable report while remaining independent of a rigid response schema.

Phase 7: Error Handling, Recovery, and Edge Cases

Phase Goal

Make the application resilient to validation problems, network failures, non-success HTTP responses, empty webhook responses, slow processing, and repeat usage.

What Will Be Implemented

- Differentiate user-input errors from backend/network errors.
- Handle non-2xx responses and empty success bodies.
- Provide Retry and Review Another File recovery actions.
- Prevent stale results and duplicate submissions.
- Keep technical details useful but user-facing messaging concise.

Copy-Paste Prompt for AI Coding Assistant

Extend the application from Phases 1-6 and harden all failure and recovery paths. Keep the existing n8n GET integration unchanged.

Implement clear error categories without exposing confusing stack traces to normal users:

Validation errors:

- Explain what is wrong with the GitHub file URL and keep the user's value available for correction.

Network/unreachable errors:

- If n8n cannot be reached, show a message that the code-review service could not be contacted and offer Retry.
- Keep enough internal/console detail for debugging, but do not dump raw exceptions into the main UI.

HTTP errors:

- Treat non-success HTTP status codes as failures even if a response body exists.
- Show a concise message including the HTTP status when useful.

Empty response:

- If the request succeeds but returns an empty or whitespace-only body, show a controlled error stating that the review service returned no review content. Do not show a fake review.

Slow request behavior:

- Keep the loading indicator stable for long-running reviews.
- Do not cycle fake backend stages that imply n8n has reached a specific internal step unless the backend

actually reports those stages.

- A simple “Review in Progress” message plus neutral helper text is sufficient.

Repeat usage:

- A second review must never show content from the previous review while the new request is loading.
- Prevent double-click/Enter from firing duplicate requests.
- Retry should retry the same validated GitHub URL intentionally.
- “Review Another File” should fully reset the page for a new URL.

If the browser blocks the call because of CORS or another localhost/browser policy, present a useful error and preserve the actual browser console/network evidence for troubleshooting. Do not conceal the problem with fake success data.

User-Facing Verification / Testing Steps

- ☐ Invalid URL: verify only validation messaging appears and no network request is sent.
- ☐ n8n stopped: verify an unreachable-service error and Retry action appear.
- ☐ Non-success response: verify the UI exits loading and shows a controlled error.
- ☐ Empty success body: verify the UI reports that no review content was returned.
- ☐ Rapidly click Review Code/press Enter: verify only one request is active.
- ☐ Complete one review, then submit a second URL: confirm the first review is cleared while the second is loading.
- ☐ Use Retry after a failure and verify the same URL is attempted again.

Expected Result

The UI handles realistic failures cleanly and always gives the user an understandable next action without misrepresenting backend state.

Phase 8: Responsive Design, Accessibility, and UX Polish

Phase Goal

Polish the working application for desktop, tablet, and mobile while improving accessibility and visual consistency without adding unrelated features.

What Will Be Implemented

- Optimize layout and result reading at multiple viewport sizes.
- Improve keyboard use, focus management, labels, ARIA/live messaging, and contrast.
- Refine buttons, icons, spacing, typography, loading indicators, and long URL/result behavior.
- Honor reduced-motion preferences.
- Keep branding and UI tokens configurable.

Copy-Paste Prompt for AI Coding Assistant

Polish the fully working application from Phases 1-7. Do not change the n8n request contract or add new backend functionality.

Responsive behavior:

- Desktop: use the available width efficiently while keeping review text at a comfortable reading measure.
- Tablet: reorganize the input and result workspace cleanly without cramped controls.
- Mobile: stack content naturally, make the primary action easy to tap, prevent long GitHub URLs/code/table content from forcing horizontal page overflow, and keep the result readable.

Accessibility:

- Ensure every form control has a real accessible label.
- Maintain clear keyboard focus indicators.
- Make all important interactions keyboard operable.
- Use semantic landmarks and heading order.
- Ensure validation and review status changes can be announced by assistive technology where practical.
- After a completed review, move focus or provide a logical focus path to the result heading without trapping the user.
- Check colour contrast for normal text, muted text, buttons, borders, errors, and success states.
- Respect prefers-reduced-motion for decorative/loading animations where possible.

UX polish:

- Refine spacing, typography, buttons, icons, cards/panels, and hover/focus/disabled states.
- Use professional developer-oriented icons only where they genuinely improve comprehension.
- Keep animations subtle and purposeful.

- Ensure long review content, code blocks, and tables can be read or scrolled safely.
- Preserve the AI Tester identity and configurable theme tokens.

Do not add unrelated features such as history, login, repository browsing, dashboards, analytics, user accounts, or GitHub authentication.

User-Facing Verification / Testing Steps

- ☐ Test at approximately 1440px desktop width, 768px tablet width, and 375px mobile width.
- ☐ Confirm the page itself does not horizontally overflow at mobile width.
- ☐ Confirm wide tables/code blocks are contained and usable without breaking the layout.
- ☐ Complete a review using keyboard-only navigation.
- ☐ Inspect focus visibility on input, Review Code, Retry, and Review Another File actions.
- ☐ Enable reduced-motion in the operating system/browser and verify nonessential motion is reduced.
- ☐ Confirm error, loading, success, and normal text remain readable with adequate contrast.

Expected Result

The application is polished, responsive, accessible, and suitable for professional demonstration as an AI Tester developer tool.

Phase 9: Final End-to-End Validation and Delivery Readiness

Phase Goal

Validate the complete interaction from user input through the existing n8n workflow and prepare a clean localhost-ready final project without changing the established architecture.

What Will Be Implemented

- Run a final functional test matrix covering valid, invalid, success, failure, reset, and repeated-review scenarios.
- Verify the network contract one final time.
- Remove mock data, dead code, debugging UI, and accidental dependencies.
- Provide concise local run instructions.
- Document only confirmed configuration values and known troubleshooting points.

Copy-Paste Prompt for AI Coding Assistant

Perform a final production-readiness pass on the GitHub Code Reviewer Agent built in the previous phases. Do not redesign it and do not change the backend contract.

Final confirmed architecture:

User -> AI Tester Custom Web UI -> validated GitHub file URL -> GET n8n production webhook -> existing n8n GitHub retrieval + LLM review workflow -> Respond to Webhook -> review text/Markdown -> safely formatted Custom UI result.

Confirm this exact integration:

- Endpoint: `http://localhost:5678/webhook/Github_code_reviewer`
- Method: GET
- Query parameter: `url`
- Authentication: none
- Frontend: plain HTML, CSS, JavaScript
- Response handling: review text/Markdown; no invented JSON schema

Run and, where appropriate, fix the following final checks:

1. Valid direct GitHub file URL completes an end-to-end review.
2. Empty/malformed/non-GitHub/repository-only URLs are rejected before the network request.
3. Loading state begins immediately and prevents duplicate submission.
4. Browser Network tools show exactly one GET request with the correct ``url`` query parameter.
5. The response comes from n8n; the frontend does not perform its own code-review analysis.
6. Returned Markdown is safely rendered and long results remain readable.

7. n8n unavailable/non-success/empty response cases produce controlled errors with recovery actions.
8. “Review Another File” fully resets the interface.
9. Two consecutive real reviews do not leak or mix state.
10. Desktop, tablet, and mobile layouts remain usable.
11. Keyboard navigation, focus visibility, labels, and status messaging remain accessible.
12. Browser console has no avoidable errors during normal successful use.

Cleanup:

- Remove temporary mock review content, integration-pending messages, development-only buttons, unused files, unused styles, and dead JavaScript.
- Keep the webhook URL in its single configuration location.
- Do not embed credentials because none are required.
- Add a short README or run section explaining exactly how to start the local static site and what localhost URL to open.
- Include a brief troubleshooting note for the case where localhost browser policy/CORS prevents the UI from reaching n8n, but do not invent a backend solution that was not requested.

Return the final project in a clean state ready for local demonstration.

User-Facing Verification / Testing Steps

- ☐ Start n8n and the frontend locally using the documented instructions.
- ☐ Open the documented localhost UI URL.
- ☐ Submit the known valid GitHub file URL and verify the complete request/review/response/result flow.
- ☐ Repeat with another valid direct GitHub file URL and confirm state isolation.
- ☐ Run invalid-input and n8n-unavailable tests.
- ☐ Check the Network tab for method, endpoint, and `url` parameter.
- ☐ Check the Console for avoidable errors.
- ☐ Test desktop, tablet, mobile, and keyboard interaction one final time.
- ☐ Confirm there is no mock review engine or direct GitHub-processing logic in the frontend.

Expected Result

A clean, localhost-ready GitHub Code Reviewer Agent exists with AI Tester branding and a verified real interaction with the existing n8n workflow.

Final Architecture Summary

Area	Final design
Frontend	Plain HTML + CSS + JavaScript, served locally.
Brand	AI Tester; professional GitHub/developer-oriented theme with centralized configurable styling.
Input	Direct GitHub file URL.
Request	GET http://localhost:5678/webhook/Github_code_reviewer with query parameter url=<GitHub file URL>.
n8n	Receives URL -> retrieves/processes GitHub code -> Basic LLM Chain + configured chat model -> code-review result -> Respond to Webhook.
Response handling	Treat successful body as review text/Markdown; safely format in the browser.
Authentication	None.
Primary UI states	Idle, validation error, review in progress, review completed, request error, reset/review another file.

Completion Definition

- The user can paste a valid direct GitHub file URL and start a review from the custom AI Tester UI.
- The frontend sends exactly one GET request to the configured n8n production webhook using the `url` query parameter.
- n8n remains responsible for GitHub code retrieval and AI review generation.
- The browser shows clear validation, loading, completion, and failure states.
- The returned review text/Markdown is rendered safely and professionally.
- The user can reset the application and review another GitHub file.
- The app runs locally and remains usable across desktop, tablet, and mobile.

End of roadmap.